

Audio Context Layer — Technical Report

Curio Next Task 1: Audio Question Answering

1. Executive Summary

This project implements a proof-of-concept Audio Context Layer that converts an audio clip into a structured temporal representation and answers natural-language questions grounded in that representation.

The core pipeline is:

Audio → 1-second temporal windows → sound-event detection + environment classification → structured audio context → question routing → grounded answer

The PoC intentionally uses a small synthetic dataset because the task explicitly permits and rewards a curated synthetic dataset. The dataset contains 90 twelve-second clips, five environment classes, five event classes, six question types, exact event timestamps, and clip-level train/validation/test splits.

On the held-out synthetic test set, the final hybrid context layer achieved:

Metric	Test result
Environment accuracy	100.0%
Environment macro-F1	1.000
Event-window accuracy	100.0%
Event-window macro-F1	1.000
QA exact-match accuracy	100.0%
QA token F1	1.000

These results should **not** be interpreted as real-world audio-QA performance. The test set follows the same controlled synthesis process as training, and the vocabulary is deliberately small. The main contribution of this PoC is the end-to-end context-layer design, reproducible data construction, temporal grounding, and evaluation framework.

2. Problem Formulation

Given an audio signal A and a natural-language question Q, the system should return an answer Y that is grounded in the acoustic evidence contained in A.

A useful intermediate representation is a temporal context:

```
Environment: park
Timeline:
3-4 s dog_bark
6-7 s dog_bark
9-10 s footsteps
```

The answer is generated from this context rather than from the question alone.

The PoC supports:

- Perceptual / apparent questions** — what environment is suggested; what sounds are present.
- Presence questions** — whether a specified event is present.

3. **Counting questions** — how many times an event occurs.
4. **Temporal questions** — what happens immediately before an event.
5. **Causal / reasoning questions** — why the environment appears to be a particular setting.
6. **Extensible routing** — the question router is isolated from the audio models so additional question types can be added.

3. Research Study

3.1 Audio Question Answering

Clotho-AQA established a dedicated audio-question-answering benchmark built from 1,991 audio files, with six questions per audio file and human-provided answers. Its experiments include binary and multi-class AQA baselines. This work supports the framing of AQA as a multimodal translation problem in which the audio signal and language question must be jointly considered.

Reference: Lipping et al., *Clotho-AQA: A Crowdsourced Dataset for Audio Question Answering*, 2022.

3.2 Audio-language representations

CLAP (Contrastive Language-Audio Pretraining) learns a shared audio/text embedding space using contrastive learning. Its zero-shot formulation is relevant to a future version of this PoC because it can map arbitrary natural-language sound concepts to audio without retraining a classifier for every label.

Reference: Elizalde et al., *CLAP: Learning Audio Concepts From Natural Language Supervision*, 2022.

3.3 Audio Spectrogram Transformer

AST applies transformer self-attention directly to audio spectrogram patches and demonstrated strong performance on AudioSet and other audio benchmarks. AST is a reasonable future replacement for the compact event classifier used in this PoC when scaling to real-world audio.

Reference: Gong, Chung & Glass, *AST: Audio Spectrogram Transformer*, 2021.

3.4 Design implication

The research suggests a separation between:

- ****acoustic representation**** — understanding what is in the audio;
- ****temporal grounding**** — knowing when it occurs;
- ****language reasoning**** — mapping a question to evidence and producing an answer.

This separation is reflected in the architecture implemented here.

4. Dataset

4.1 Construction

A synthetic dataset was generated specifically for this PoC. Each sample is:

- 12 seconds long;
- mono;

- 16 kHz sampling rate;
- composed of an environment background plus 2–4 inserted sound events;
- annotated with exact event start/end times;
- paired with six natural-language questions.

The environment classes are:

- home
- office
- street
- park
- rain

The event classes are:

- door_knock
- dog_bark
- clap
- bell
- footsteps

The generator creates controlled acoustic signatures for each class using deterministic waveform construction, filtered noise, harmonic components, and decaying transients. Randomization changes event ordering, event counts, and background noise while preserving the ground-truth annotation.

4.2 Question types

Each clip receives six questions:

Type	Example
what	What environment does this audio suggest?
what_sound	What sounds are present in the audio?
presence	Is there a dog bark in the audio?
counting	How many times does a dog bark occur?
temporal	What happens immediately before footsteps?
why	Why does the audio appear to be in this environment?

4.3 Split

The split is performed at the **clip level**, not at the window level, preventing windows from the same audio clip from appearing in multiple splits.

Split	Clips	Approx. proportion
Train	62	68.9%
Validation	13	14.4%
Test	15	16.7%

Split	Clips	Approx. proportion
Total	90	100%

The test set contains 90 QA pairs (15 clips × 6 questions).

4.4 Data format

Each split is stored as JSON. A record contains:

```
{
  "clip_id": "clip_0008",
  "audio_path": "data/audio/clip_0008.wav",
  "environment": "park",
  "events": [
    {"event": "dog_bark", "start": 3, "end": 4},
    {"event": "dog_bark", "start": 6, "end": 7},
    {"event": "footsteps", "start": 9, "end": 10}
  ],
  "qa": [...]
}
```

5. Method

5.1 Audio preprocessing

The audio is converted into log-mel spectrogram features using:

- 16 kHz sampling rate;
- 64 mel bands;
- 512-point FFT;
- 160-sample hop length;
- frequency range 40–7600 Hz.

5.2 Temporal event layer

The audio is divided into twelve one-second windows.

For each window, compact statistics are extracted from the log-mel spectrogram:

- mean per mel band;
- standard deviation per mel band;
- maximum per mel band;
- waveform mean absolute amplitude;
- waveform standard deviation.

A balanced Random Forest classifier predicts one of six labels: five event classes plus background.

This design was selected because the PoC dataset is intentionally small. A compact classifier is faster to train and easier to reproduce than fine-tuning a large audio transformer.

5.3 Environment layer

The complete twelve-second clip is converted into the same log-mel representation. Mean and standard-deviation statistics are passed to a Random Forest classifier predicting the five environment classes.

5.4 Learned CNN baseline

A lightweight convolutional neural network was also implemented and trained on one-second log-mel spectrograms. Its purpose is to provide a learned deep baseline and satisfy the requirement to demonstrate a trainable audio model and loss curves.

The final context-layer inference uses the compact Random Forest detector because it produced more stable results on this deliberately small synthetic dataset. The CNN remains in the repository as a reproducible baseline and its training history/loss curve is stored under `results/`.

5.5 Context representation

The two model outputs are merged into a structured object:

```
{
  "environment": "park",
  "environment_confidence": 0.99,
  "timeline": [
    {"start": 3, "end": 4, "event": "dog_bark", "confidence": 1.0},
    {"start": 6, "end": 7, "event": "dog_bark", "confidence": 1.0},
    {"start": 9, "end": 10, "event": "footsteps", "confidence": 0.98}
  ]
}
```

This structured context is the central Audio Context Layer.

5.6 Question routing

The router identifies the question type using lightweight linguistic patterns and extracts an event target where required.

Examples:

- `How many times...` → counting
- `Is there...` → presence
- `What happens immediately before...` → temporal
- `Why does...` → causal
- `What sounds are present...` → perceptual sound listing
- `What environment...` → environment perception

Answers are generated only from detected context, keeping the PoC grounded.

6. Experimental Setup

Hardware/software requirements are intentionally modest. The implementation uses Python, NumPy, SciPy, librosa, scikit-learn, PyTorch, SoundFile, Matplotlib and Flask.

The complete experiment is reproducible with:

```
python -m src.generate_dataset
python -m src.train
python -m src.evaluate
```

A direct inference example is:

```
python -m src.demo \
--audio data/audio/clip_0008.wav \
--question "How many times does a dog bark occur?"
```

Example result:

ANSWER: 2

Environment: park (0.99)
3-4s: dog_bark (1.00)
6-7s: dog_bark (1.00)
9-10s: footsteps (0.98)

7. Results

7.1 Model-level results

Component	Accuracy	Macro-F1
Environment classifier	1.000	1.000
Event-window classifier	1.000	1.000

7.2 QA results

Question type	N	Exact Match	Token F1
What / environment	15	1.000	1.000
What / sounds	15	1.000	1.000
Presence	15	1.000	1.000
Counting	15	1.000	1.000
Temporal	15	1.000	1.000
Why / causal	15	1.000	1.000
Overall	**90**	**1.000**	**1.000**

7.3 Loss curve

The repository contains the CNN training loss curve at:

results/event_model_loss.png

The CNN validation accuracy varied across epochs and reached approximately 84% at its best validation checkpoint. This is included as a learned baseline rather than the final detector.

8. Error Analysis

The final synthetic test evaluation produced no exact-match errors. This should be interpreted cautiously: the dataset is generated from the same controlled process used to create training examples, and the sound vocabulary is intentionally small.

The earlier CNN baseline showed errors concentrated in acoustically similar or lower-energy classes such as door knocks, footsteps, and claps. This motivated the final compact spectral-statistics classifier for the PoC.

A more realistic error analysis for the next iteration should include:

- real environmental recordings;
- overlapping events;

- unseen sound classes;
- reverberation and microphone variation;
- noisy speech/questions;
- events occurring between fixed one-second boundaries;
- semantically equivalent but lexically different answers.

9. Observations and Limitations

Strengths

1. Complete end-to-end pipeline from audio to natural-language answer.
2. Explicit temporal grounding rather than treating the clip as a single undifferentiated vector.
3. Synthetic dataset is fully reproducible and includes exact event timestamps.
4. Dataset is split by clip to avoid temporal-window leakage.
5. Multiple question types are evaluated separately.
6. Training, inference, evaluation and API entry points are included.

Limitations

1. The dataset is synthetic and small.
2. The vocabulary contains only five event classes and five environments.
3. The question router is rule-based rather than a general-purpose language model.
4. The causal answers are template-based and should not be interpreted as learned causal inference.
5. The one-second segmentation limits temporal precision and can fail when events overlap or straddle boundaries.
6. The perfect test score is a property of the controlled synthetic benchmark, not evidence of general audio understanding.
7. The final event detector uses engineered spectral statistics and a Random Forest rather than a large pretrained audio-language model.

10. Recommended Production / Research Extension

The natural next step is to replace the controlled classifier stack with a pretrained audio-language encoder such as CLAP or an Audio Spectrogram Transformer, while retaining the same context-layer abstraction.

A scalable architecture would be:

```

Audio
↓
Voice / sound-event segmentation
↓
Pretrained audio encoder (CLAP / AST / PANN-style model)
↓
Timestamped event candidates + embeddings
↓
Temporal Context Store
↓
Question encoder / LLM

```

↓
Evidence retrieval from context
↓
Grounded answer + supporting timestamps

For a larger dataset, the context store could expose evidence such as:

Evidence 1: dog_bark, 3.1–3.8s, confidence 0.91
Evidence 2: dog_bark, 6.0–6.7s, confidence 0.88
Evidence 3: footsteps, 9.0–9.9s, confidence 0.93

The answering model can then be constrained to cite those evidence spans, reducing unsupported answers.

11. Repository Structure

```
curio_audio_context/  
  README.md  
  requirements.txt  
  data/  
    audio/  
      train.json  
      val.json  
      test.json  
      dataset_meta.json  
    models/  
      event_rf.joblib  
      environment_model.joblib  
      event_model.pt  
    results/  
      metrics.json  
      qa_predictions.json  
      event_model_history.json  
      event_model_loss.png  
    src/  
      generate_dataset.py  
      audio_utils.py  
      model.py  
      train.py  
      context_layer.py  
      evaluate.py  
      demo.py  
      api.py  
    docs/  
      TECHNICAL_REPORT.md
```

12. References

1. Samuel Lipping, Parthasaarathy Sudarsanam, Konstantinos Drossos, Tuomas Virtanen. *Clotho-AQA: A Crowdsourced Dataset for Audio Question Answering*. 2022.
2. Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, Huaming Wang. *CLAP: Learning Audio Concepts From Natural Language Supervision*. 2022.
3. Yuan Gong, Yu-An Chung, James Glass. *AST: Audio Spectrogram Transformer*. 2021.
4. Alec Radford et al. *Robust Speech Recognition via Large-Scale Weak Supervision*. 2022. (Relevant for future speech/transcription integration.)

13. Submission Statement

This submission deliberately prioritizes a complete, reproducible PoC over an oversized model. The system demonstrates dataset creation, model training, temporal context construction, question routing, quantitative

evaluation, loss visualization, error analysis, and documentation. The limitations are explicitly documented so that the reported synthetic results are not presented as evidence of general real-world audio-QA capability.